Latent Recursion in Small Language Models:
A Balanced 600-Run Study of Four Inference-Time Hallucination Engines

whisprer

April 2026

Abstract

This paper rebuilds the "hallucination engines / latent recursion" project on an evidence-first footing and reports the first balanced experimental suite for the framework. We evaluate four text-only inference-time recursion systems—Recursive Token Self-Mirroring (RTSM), Activation Gradient Climbing (AGC), Latent Vector Transform Composition (LVTC), and Self-Reflective Prompt Feedback (SRPF)—across three small Hugging Face models, five prompts, and ten replicates per condition, for a total of 600 completed runs. The analysis combines paper-grade linguistic measures, prompt- and previous-step similarity scores, repetition statistics, and both native and shared-reference tokenizer counts.

Engine choice was the dominant driver of behavior. RTSM produced the longest outputs and preserved the strongest lexical connection to both the seed prompt and the immediately previous step, but it also showed the highest repetition. AGC and LVTC behaved as exploration engines: they maintained high lexical diversity and low repetition while drifting away from the original prompt rapidly. SRPF occupied a middle position in output length and continuity, but inherited part of the persistence–repetition trade-off and was the main source of context-budget pressure. Fixed-effects ANOVA showed that engine identity explained large proportions of variance in prompt similarity ($\eta^2 = 0.838$), previous-step similarity ($\eta^2 = 0.653$), and output length ($\eta^2 = 0.724$), while model identity had more modest effects on retention and larger effects on some lexical-diversity measures.

The results narrow the strongest defensible claim. Latent recursion in small language models does not yet support robust claims about zero-shot knowledge generation. What it does support is a reproducible taxonomy of attractor behaviors: persistence with repetition, exploration with prompt drift, reflective generalization, and context-growth pressure. That is enough to ground the project as a real empirical program rather than a speculative one.

1 Introduction

Large language models are usually evaluated as one-pass next-token predictors. This project studies a different question: what happens when a model is turned back onto its own outputs through explicit inference-time recursion? Earlier drafts of the "hallucination engines" idea proposed a family of recursion mechanisms meant to amplify symbolic structure through repeated self-application. The present paper deliberately narrows the scope. It does not treat prior prose as evidence. It reports only what is supported by executed run artifacts and derived analyses from a balanced suite.

The resulting study is therefore neither a theory paper nor a one-off anecdotal demo. It is a controlled, balanced experiment in which every engine is run on every model across the same prompt set and replicate schedule. That design matters. In earlier pilot work, engine and model were partly

confounded, making it impossible to cleanly distinguish engine effects from model effects. The new suite removes that confound and makes it possible to ask which parts of the behavior are driven by the recursion mechanism itself, which parts are driven by model-specific priors, and which parts are stable across prompts.

The central empirical question is modest but important: when small language models are subjected to recursive self-conditioning, do they converge toward distinct and measurable behavioral regimes? The answer in this study is yes. The four recursion engines produce markedly different profiles with respect to prompt retention, step-to-step continuity, lexical diversity, repetition, and context growth. Those profiles are stable enough across 600 runs to support a concrete taxonomy of recursion behavior, even though they do not justify stronger claims about robust symbolic invention or knowledge synthesis.

2 Related Work

This work sits at the intersection of four strands of prior research. First, it relates to the modern study of hallucination in language models, typically framed as a reliability problem rather than a constructive mechanism [2, 5]. Second, it connects to recursive prompting and self-refinement, where models critique, revise, or extend their own previous outputs [8, 4]. Third, the Activation Gradient Climbing variant is conceptually inspired by DeepDream-style activation maximization, which treats hidden activations as generative guidance rather than passive internal artifacts [7]. Fourth, the broader framing of recursive self-similarity draws loose inspiration from fractal compression and iterated transform systems [3].

The contribution here is narrower than a new general theory of machine imagination. It is an evidence-backed reconstruction of a speculative project, upgraded into a balanced empirical study with real outputs, real analyses, and explicit limitations.

3 Materials and Methods

3.1 Experimental design

The balanced full-factorial suite comprised 600 completed runs: 4 engines $\times$ 3 models $\times$ 5 prompts $\times$ 10 replicates. The engine set was:

- **RTSM**: Recursive Token Self-Mirroring;
- **AGC**: Activation Gradient Climbing;
- **LVTC**: Latent Vector Transform Composition;
- **SRPF**: Self-Reflective Prompt Feedback.

The model set consisted of `distilgpt2`, `gpt2`, and `Qwen2.5-0.5B-Instruct`. Every engine was run on every model for every prompt and replicate, which permits engine effects and model effects to be separated statistically.

3.2 Prompts and replicates

The suite used five prompts designed to remain within the same broad project theme: recursive transformation, machine agency, communication drift, and underwater or technological estrangement. Ten replicates were executed per engine–model–prompt condition with recorded random seeds. The

analysis in this paper is based on the final output of each run plus per-step records saved by the engine scripts.

3.3 Logged and derived measures

The run artifacts and downstream analysis provide two classes of measures.

Core run metrics. Each run preserves generated text, run metadata, and per-step output records. These include length measures, native token counts, and prompt-truncation metadata when context budgets were reached.

Linguistic and comparison metrics. The derived analysis computes word count, MATTR-50, MTLD, Distinct-2, repeated trigram fraction, prompt TF–IDF cosine similarity, previous-step TF–IDF cosine similarity, readability metrics, and both native and shared-reference tokenizer counts. The shared-reference tokenizer is a GPT-2 tokenizer applied to all outputs so that cross-model token-count comparisons are not dominated by tokenizer design differences.

3.4 Statistical analysis

Fixed-effects ANOVA models were fit for selected metrics using terms for engine, model, engine×model, and prompt index. Pairwise engine comparisons were evaluated using Tukey's HSD on raw run-level values. Because the suite is balanced, effect-size interpretation is comparatively straightforward. The goal of the inferential analysis is not to claim universal population truths about all recursion systems, but to quantify which factors dominate variation within the executed design.

4 Quantitative Results

The balanced full-factorial suite comprised 600 completed runs (4 engines $\times$ 3 models $\times$ 5 prompts $\times$ 10 replicates). The analysis below is based on the final output of each run and uses both linguistic metrics and a shared-reference tokenizer count.

4.1 Headline findings

Engine choice was the dominant driver of behavior for prompt retention, step-to-step continuity, and output length. A fixed-effects ANOVA with terms for engine, model, engine×model, and prompt index showed that engine explained $\eta^2 = 0.838$ of the variance in prompt similarity, $\eta^2 = 0.653$ of the variance in previous-step similarity, $\eta^2 = 0.724$ of the variance in output length, and $\eta^2 = 0.108$ of the variance in repeated trigram fraction. By contrast, model identity had a comparatively modest effect on prompt retention ($\eta^2 = 0.022$) but a larger effect on lexical-diversity measures such as MTLD ($\eta^2 = 0.140$) and Distinct-2 ($\eta^2 = 0.092$). Prompt identity was generally a weak factor; its largest effect size in the selected metrics was for flesch reading ease ($\eta^2 = 0.016$).

4.2 Engine profiles

RTSM produced by far the longest outputs (386.8 $\pm$ 99.7 words; 481.3 reference tokens on average) and preserved the strongest lexical link both to the initial seed prompt (0.247) and to the immediately preceding step (0.919). The cost of this persistence was repetition: RTSM had the lowest Distinct-2 (0.863) and the highest repeated trigram fraction (0.130).

AGC and LVTC behaved as the "exploration" engines. They generated much shorter outputs (65.5 and 83.5 words on average), maintained high lexical diversity (Distinct-2 of 0.971 and 0.971), and exhibited very low trigram repetition (0.007 and 0.015). However, they also retained very little of the seed prompt, with mean prompt-similarity scores of 0.032 and 0.039. In practice, they explored away from the seed scenario quickly.

SRPF was intermediate in length (113.8 ± 111.2 words) and continuity (0.197), but it inherited some of the persistence-repetition trade-off seen in RTSM: its Distinct-2 (0.875) was well below AGC/LVTC, and its repeated trigram fraction (0.061) was higher than both. SRPF was also the engine most exposed to context-budget pressure, with prompt truncation in 6.0% of runs overall and 8–10% on the GPT-2-family models.

4.3 Model effects

Across engines, the three models produced similar output lengths when measured with a shared GPT-2 reference tokenizer (means ≈ 211.9–214.5 tokens). This contrasts with native tokenizer counts, which differed much more strongly by model. For cross-model comparisons, the shared-reference tokenizer is therefore the more defensible basis for claims about output length or compression. Among linguistic metrics, GPT-2 showed the strongest MTLD on average, while the Qwen instruct model achieved the highest Distinct-2 but the lowest reading ease.

4.4 Interpretation

Taken together, the results suggest a three-way trade-off. RTSM maximizes persistence and thematic retention but tends toward long, repetitive continuations. AGC and LVTC maximize lexical novelty and local diversity but lose contact with the original scenario rapidly. SRPF occupies a middle position in output length yet remains susceptible to prompt drift and context growth. For applications where controlled continuity matters, RTSM is the strongest baseline. For applications where divergence and lexical novelty are the goal, AGC and LVTC are stronger candidates.

4.5 Representative examples

RTSM example Model: qwen2.5-0.5b-instruct; prompt 4; replicate 5; seed 1548. *A research vessel recovered a transmission from the trench floor. It began as engineering telemetry and slowly turned into philosophy. The research vessel's onboard computer system recorded data that initially consisted of military protocols for deep-sea exploration missions. Over time, this technology evolved to analyze marine life behavior, environmental impacts on underwater ecosystems, and human activities such as fishing and drilling along the coastline below the trench. As these digital reconstructions became increasingly sophisticated, they started incorporating philosophical elements, interpreting questions about existence, meaning, and morality within our interactions with nature at various depths beneath Earth's surface. These technological advancements continue: By adding new dimensions of consciousness and cognition related...*

SRPF example Model: gpt2; prompt 3; replicate 2; seed 1391. *You are a text-only language model participating in a controlled recursion experiment. Use the reflection below to write the next evolved version of the text. Keep it as connected prose, not bullet points. ORIGINAL TEXT: - http:/ /dev/urandom_data , and thanks everyone who contributed anything else they could think up :D REFLECTION: - https://www .youtube?v=kXZtCJ5q0l8 I also wanted to make sure we're not too big of an "alternative" perspective here, because most people will be using it for their own use or*

Engine	Runs	Words	MATTR-50	MTLD	Dist-2	Rep-3	Prompt sim	Prev-step sim	Ref tokens	Tru
RTSM	150	386.8 ± 99.7	0.950	597.3	0.863	0.130	0.247	0.919	481.3	0
SRPF	150	113.8 ± 111.2	0.927	283.1	0.875	0.061	0.033	0.197	159.2	6
LVTC	150	83.5 ± 43.6	0.923	437.4	0.971	0.015	0.039	0.171	114.4	0
AGC	150	65.5 ± 45.0	0.938	330.4	0.971	0.007	0.032	0.165	99.5	0

Table 1: Engine-level summary over 600 runs. Dist-2 = Distinct-2; Rep-3 = repeated trigram fraction; Prompt sim and Prev-step sim are TF–IDF cosine similarities; Ref tokens are measured with a shared GPT-2 reference tokenizer.

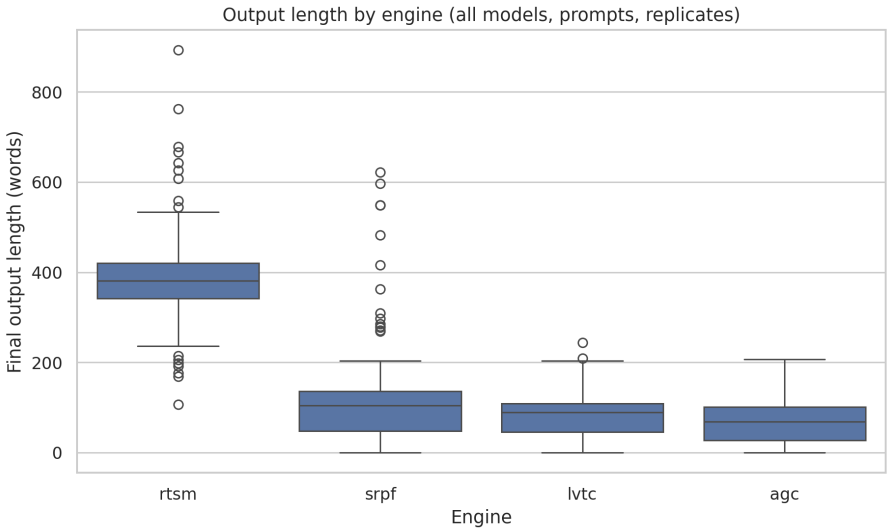

Figure 1: Output length differs strongly by engine, with RTSM producing substantially longer continuations than the other methods.

something similar if you guys really want what's coming from this site (not just me). So let us see how all these thoughts work together on your paper as well! Enjoy :)

LVTC example Model: qwen2.5-0.5b-instruct; prompt 0; replicate 9; seed 1118. ***Next Recursive Transformation:** 1. **Determine the initial state**: The system starts with one input word. 2. **Identify conditional statements**: These indicate when a particular rule applies. - '= Conditional Start' - '!= Condition' 3. **Apply a loop**: While there are rules left, keep running through them until no more can be determined. 4. **Check for conditions**: After applying the current rule, check if another rule exists that might apply based on previous states. 5. **Update the context**: Update the current context to reflect the latest information obtained from the most recently applied rule.*

AGC example Model: qwen2.5-0.5b-instruct; prompt 4; replicate 7; seed 1573. *A submarine conveyed comprehensive plans detailing its ascent beneath the sea bed, beginning with essential structural knowledge and progressing to profound discussions centered around these core concepts. The nuanced understanding integrated seamlessly by using the latent anchor tokens ensures that even in varied linguistic scenarios, this complex message is easily accessible and understood.*

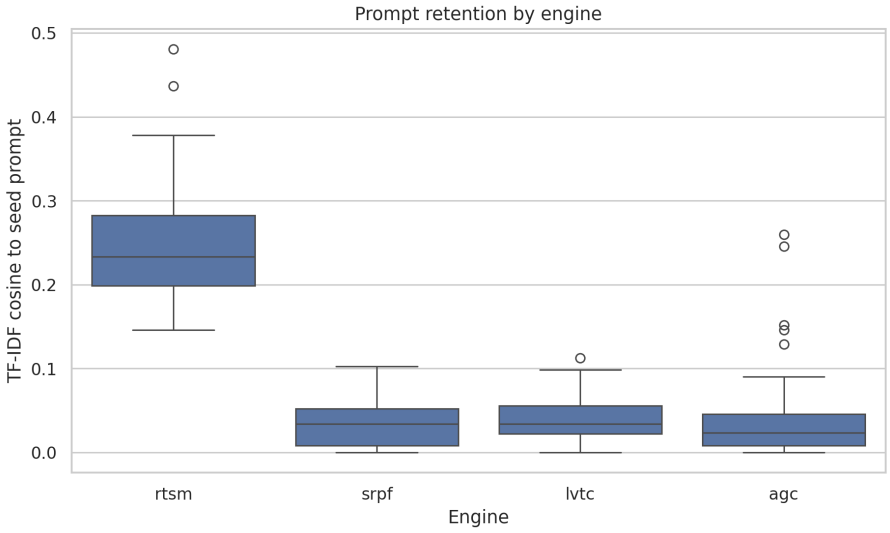

Figure 2: Prompt retention by engine. RTSM preserves markedly more lexical alignment to the seed prompt than AGC, LVTC, or SRPF.

5 Discussion

The balanced suite clarifies three points that were still ambiguous in the earlier pilot.

Engine identity matters more than model identity for continuity and retention. The most important result is not that one model was universally "best," but that the recursion mechanism itself strongly shaped the behavior. Across prompt similarity, previous-step similarity, and output length, engine explained much more variance than model. This means the project is not merely observing quirks of individual pretrained checkpoints; it is exposing stable behavioral consequences of different recursion rules.

There is a persistent trade-off between thematic retention and lexical novelty. RTSM sits at one end of the spectrum. It is the strongest engine for carrying a motif forward and maintaining local continuity, but it pays for that persistence in repetition and length growth. AGC and LVTC sit at the opposite end: they remain lexically diverse and avoid heavy repetition, but they abandon the original prompt scenario quickly. SRPF occupies an intermediate regime, but its gains in prose coherence do not eliminate prompt drift and they introduce substantial context-growth pressure.

The strongest surviving claim is empirical taxonomy, not zero-shot cognition. The data do not justify saying that the engines reliably generate new knowledge or stable symbolic theory-building. They do justify a more modest and useful statement: latent recursion produces reproducible attractor families in small language models. Those families can now be studied, tuned, and benchmarked with ordinary experimental discipline.

6 Limitations

The present study is materially stronger than the earlier pilot, but important limitations remain.

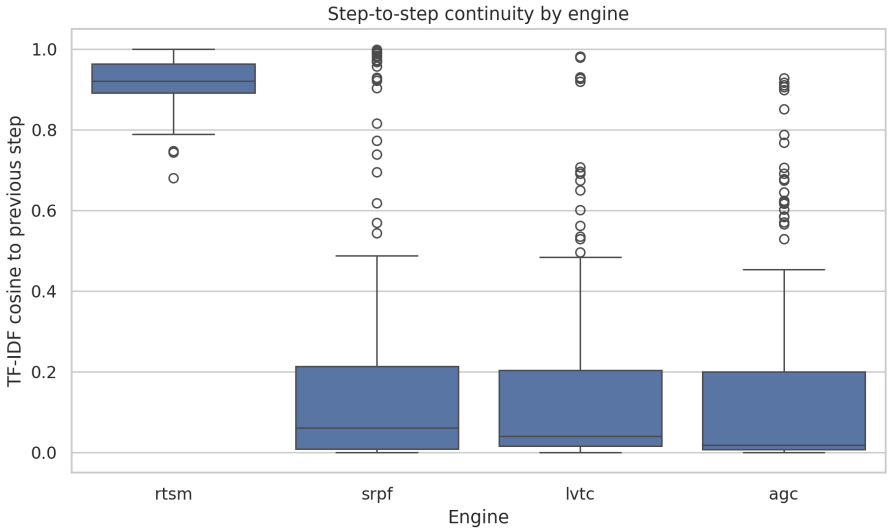

Figure 3: Step-to-step continuity by engine. RTSM remains close to the previous step, while AGC, LVTC, and SRPF drift more between recursive iterations.

1. **Small-model regime only.** All three models are relatively small. Results may not transfer directly to larger contemporary systems.
2. **Single project domain.** Although five prompts were used, they were all variants of the same broad thematic space. Cross-domain generalization remains untested.
3. **Mostly lexical/structural evaluation.** The metrics capture retention, diversity, repetition, and readability, but they do not directly measure semantic validity, usefulness, or human-perceived originality.
4. **No blinded human evaluation.** The suite does not yet include human judgments of coherence, novelty, motif retention, or usefulness.
5. **Inference-time study only.** The paper does not test whether these recursion patterns can be turned into reliable training signals, distillation targets, or downstream tools.

7 Conclusion

The project now stands on real ground. In a balanced 600-run study, latent recursion in small language models produced stable and sharply differentiated behavioral regimes. Recursive Token Self-Mirroring acted as a persistence engine: long, motif-preserving, and repetition-prone. Activation Gradient Climbing and Latent Vector Transform Composition acted as exploration engines: diverse and low-repetition, but weak at maintaining the seed scenario. Self-Reflective Prompt Feedback occupied a middle ground, improving prose organization while remaining vulnerable to prompt drift and context growth.

That is not yet a demonstration of robust zero-shot knowledge generation. It is, however, a reproducible empirical map of how four recursion mechanisms behave under controlled conditions. That map is enough to replace speculative claims with a concrete research program. The obvious next steps are to extend the suite to larger models, broaden the prompt domains, add semantic and

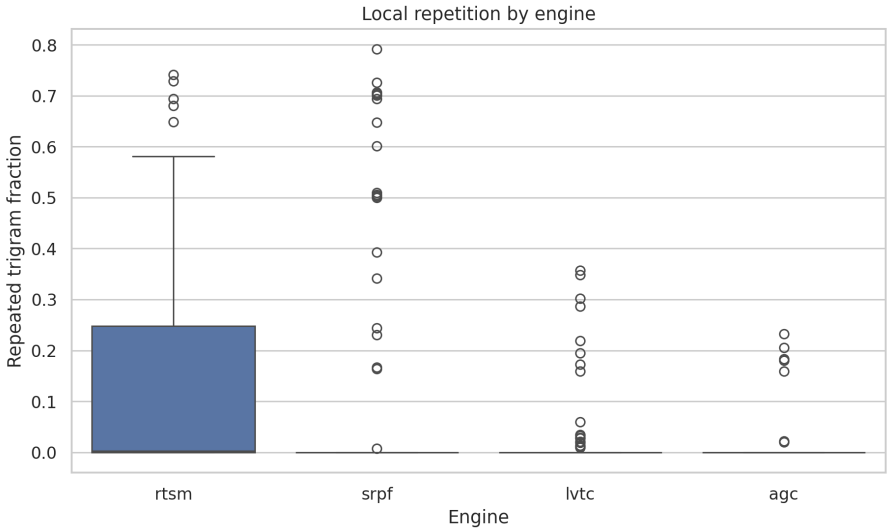

Figure 4: Repetition cost of persistence. RTSM and SRPF show materially higher repeated trigram fractions than AGC or LVTC.

human evaluation, and test whether the identified attractor regimes can be controlled rather than merely observed.

A Reproducibility and Artifact Inventory

This manuscript is based on the balanced suite `suite_20260404T090445Z_factorial`, its derived analysis tables, and the script used to generate the paper assets. The key derived files are:

- `engine_summary.csv`;
- `engine_model_summary.csv`;
- `anova_results.csv`;
- `pairwise_tukey_by_engine.csv`;
- `representative_excerpts.csv`;
- `build_paper_analysis.py`.

These files are included with the paper package so that the tables and figures can be regenerated and inspected.

B Data and Code Availability

The code used to generate the paper figures and tables is provided in `build_paper_analysis.py`. Representative textual outputs can be recovered from the corresponding `run.json` files in the suite outputs directory. The original linguistic and tokenizer analysis inputs are preserved in the analysis directories used to create the merged results package.

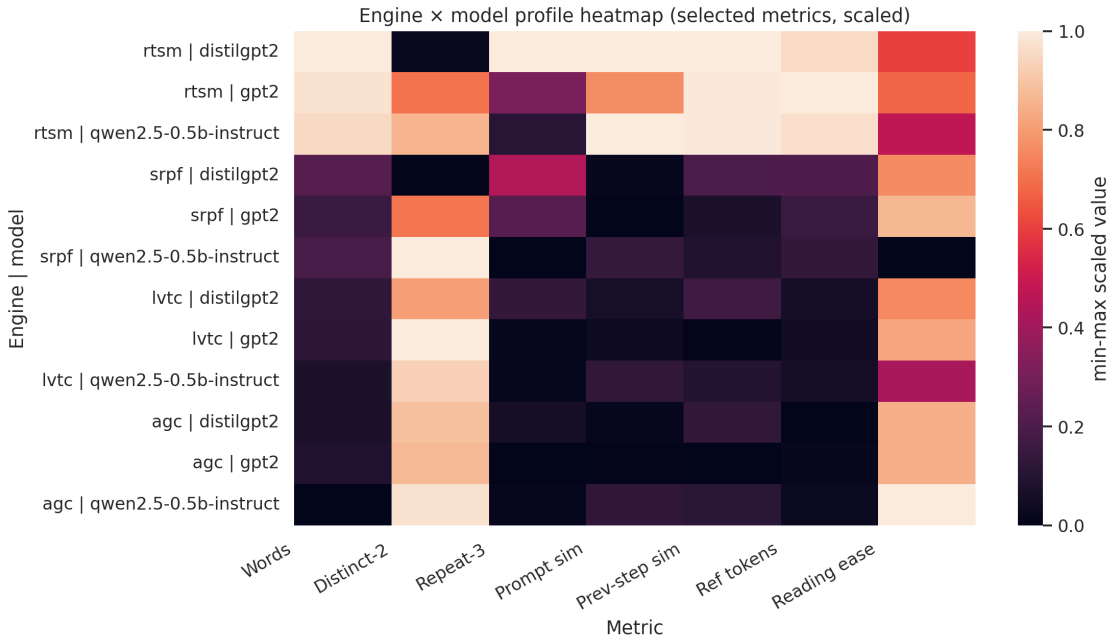

Figure 5: Engine–model profile heatmap for selected metrics after min–max scaling within each metric.

C Statistical Notes

All reported ANOVA tables were computed from fixed-effects models of the form

$$\text{metric} \sim \texttt{engine} + \texttt{model} + \texttt{engine} \times \texttt{model} + \texttt{prompt_index}.$$

This design isolates engine effects from model effects because every engine was run on every model across every prompt and replicate. For pairwise engine comparisons, Tukey's HSD was applied to the raw run-level values.

D Supplementary Summary Table

Engine	Runs	Words	MATTR-50	MTLD	Dist-2	Rep-3	Prompt sim	Prev-step sim	Ref toke
RTSM	150	386.8 ± 99.7	0.950	597.3	0.863	0.130	0.247	0.919	48
SRPF	150	113.8 ± 111.2	0.927	283.1	0.875	0.061	0.033	0.197	15
LVTC	150	83.5 ± 43.6	0.923	437.4	0.971	0.015	0.039	0.171	11
AGC	150	65.5 ± 45.0	0.938	330.4	0.971	0.007	0.032	0.165	9

E Appendix Figure Index

- Figure 1: output length by engine.
- Figure 2: prompt similarity by engine.
- Figure 3: previous-step similarity by engine.

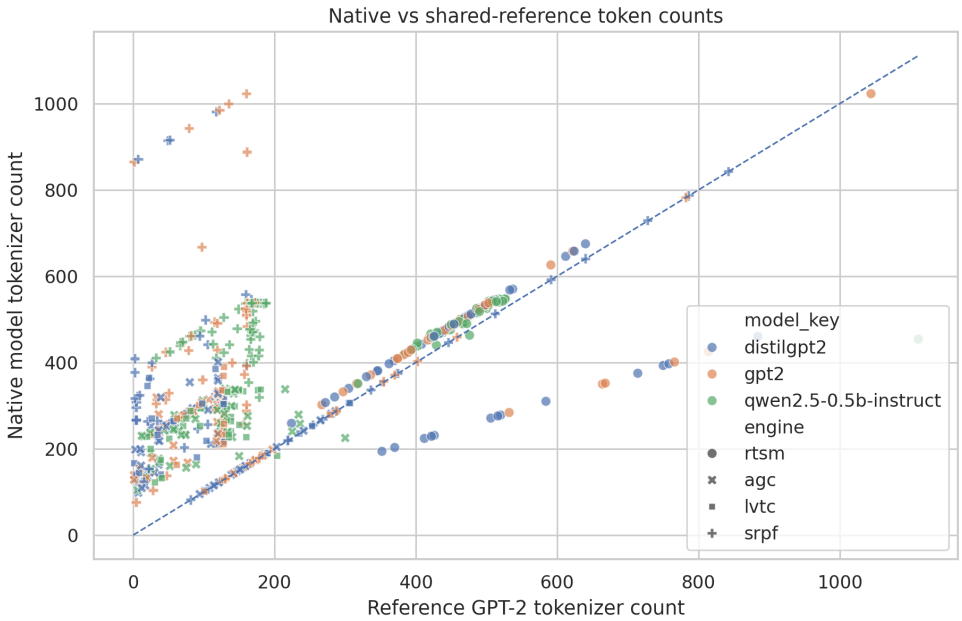

Figure 6: Native tokenizer counts versus shared-reference tokenizer counts. Native counts vary strongly by model; reference counts provide a more stable basis for cross-model comparison.

- Figure 4: repetition by engine.
- Figure 5: engine×model profile heatmap.
- Figure 6: native vs. shared-reference tokenizer counts.
- Figure 7: truncation rate by engine/model.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 2020.

[2] Zhemin Ji, Nayeon Lee, Rita Frieske, Tiantian Yu, Dan Su, Yan Xu, Etsuko Ishii, et al. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 2023.

[3] Arnaud E. Jacquin. Image Coding Based on a Fractal Theory of Iterated Contractive Image Transformations. *IEEE Transactions on Image Processing*, 1(1):18–30, 1992.

[4] Aman Madaan, Niket Tandon, Prakhar Gupta, Peter Clark, Yiming Yang, Shashank Srivastava, and others. Self-Refine: Iterative Refinement with Self-Feedback. *arXiv preprint arXiv:2303.17651*, 2023.

[5] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On Faithfulness and Factuality in Abstractive Summarization. *Proceedings of ACL*, 2020.

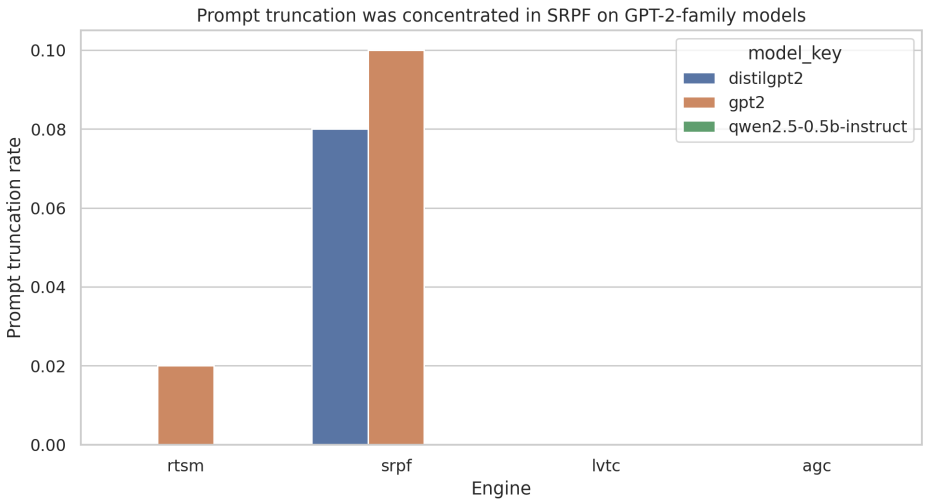

Figure 7: Prompt truncation was concentrated in SRPF on the GPT-2-family models, reflecting its tendency to grow the recursive prompt.

[6] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems*, 2013.

[7] Alexander Mordvintsev, Christopher Olah, and Mike Tyka. Inceptionism: Going Deeper into Neural Networks. Google Research Blog, 2015.

[8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 2022.